PERMISOS Y

USUARIOS

chmod · chown · umask · Grupos · Seguridad base

Repaso Clase 1:

Kernel vs GNU · Filosofía Unix · FHS · Comandos pwd ls cd mkdir touch cp mv rm cat grep

CTT - Técnico en Redes y Software · Prof. Juan Damián Pajares García · Semana 2

✓ Entender el modelo de seguridad de Linux (usuarios, grupos, permisos) ✓ Usar chmod, chown, umask con criterio profesional

Permisos Unix

rwxrwxrwx

```
$ ls -la  →  drwxr-xr-x  2  juan  juan  4096
```

01 // El modelo de Usuarios y Grupos

Linux es multiusuario desde su origen. Cada proceso, archivo y recurso tiene un dueño.

UID 0 root

- Superusuario — control total del sistema
- UID 0: el único número que importa
- Acceso irrestricto a cualquier archivo
- sudo permite escalar privilegios temporalmente
- Principio: nunca trabajar como root por defecto

UID 1000+ Usuarios

- Cada persona o servicio tiene su cuenta
- Home propio: /home/usuario
- UID ≥ 1000 en sistemas modernos
- Cuentas de servicio: www-data, mysql, nobody
- Almacenados en /etc/passwd y /etc/shadow

GID Grupos

- Agrupan usuarios con permisos compartidos
- Grupo primario: asignado en la creación
- Grupos secundarios: acceso a recursos extra
- Ej: grupo docker, sudo, www-data, disk
- Almacenados en /etc/group

"En Linux, todo tiene un dueño. La seguridad se construye sobre ese principio."

02 // Las bases de datos del sistema

Todo usuario existe en estos archivos. Conocerlos es entender cómo Linux identifica a cada proceso.

📄 /etc/passwd — Información de cuentas (públi

Legible por todos los usuarios. Contraseñas reales ya NO están aquí (solo 'x').

usuario	:	x	:	1000	:	1000	:	Juan D.	:	/home/juan	:	/bin/bash
nombre		pass→shadow		UID		GID		GECOS		home		shell

```
juan:x:1000:1000:Juan D.:/home/juan:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

🔒 /etc/shadow — Contraseñas (root only)

Solo root puede leer este archivo. Contiene el hash de la contraseña (SHA-512) y políticas de expiración.

```
juan:$6$XsR...:19845:0:99999:7:::
```

👥 /etc/group — Definición de grupos

```
nombre:contraseña:GID:miembros

sudo:x:27:juan,admin
docker:x:999:juan,deploy
```

Comandos clave:

```
$ cat /etc/passwd | grep $USER
$ sudo cat /etc/shadow | head -5
$ getent group sudo
$ id juan
$ whoami
```

"El archivo /etc/shadow es la primera línea de defensa de un sistema Linux." — Hardening básico

03 // Leyendo permisos — ls -la

Antes de cambiar permisos, tenés que entender qué estás leyendo.

```
$ ls -la /home/juan/
-   rwx   r-x   r--   2   juan   juan   4096   Mar 18 10:22   proyecto/
```

Tipo	Dueño	Grupo	Otros	Hard links	Propietario	Grupo	Tamaño	Fecha modif.	Nombre
- archivo d directorio l	Permisos del propietario (owner)	Permisos de miembros del grupo	Permisos del resto del mundo	Nº de enlaces del inode	Usuario dueño del archivo	Grupo dueño del archivo	Bytes (dirs = 4096 por bloque)	Última modif. del contenido	Nombre del archivo o directorio

Tabla de bits rwx

Bit	#	En archivo	En directorio
r	4	Leer contenido del archivo	Listar contenido del directorio
w	2	Modificar / escribir archivo	Crear / eliminar archivos dentro
x	1	Ejecutar como programa/script	Acceder / entrar al directorio (cd)

Ejemplos del mundo real

-rwxr-xr-x	Ejecutable público (/bin/ls)
-rw-r--r--	Config pública (/etc/hostname)
-rw-------	Clave privada SSH (~/.ssh/id_rsa)

"Si no entendés ls -la, no entendés la seguridad de Linux."

04 // chmod — Cambiar permisos

Dos sintaxis para el mismo comando. Los profesionales usan ambas según el contexto.

Modo OCTAL — Rápido y preciso

Cada dígito = suma de bits activos del grupo (owner/group/others).

7	r w x	= 4+2+1	Todo permitido
6	r w -	= 4+2+0	Leer y escribir
5	r - x	= 4+0+1	Leer y ejecutar
4	r - -	= 4+0+0	Solo lectura
0	- - -	= 0+0+0	Sin permisos

```
$ chmod 755 script.sh
```
→ rwxr-xr-x — ejecutable público

```
$ chmod 644 config.txt
```
→ rw-r--r-- — archivo de configuración

```
$ chmod 600 id_rsa
```
→ rw------- — clave SSH privada

Modo SIMBÓLICO — Granular y legible

Ideal para modificar permisos específicos sin afectar el resto.

```
chmod [ugoa] [+-=] [rwx] archivo
```

u	user (propietario)
g	group (grupo)
o	others (resto)
a	all (todos u+g+o)
+	agregar permiso
-	quitar permiso
=	establecer exacto

```
$ chmod u+x deploy.sh
```
→ Agrega ejecución al dueño

```
$ chmod g-w archivo.txt
```
→ Quita escritura al grupo

```
$ chmod o= secreto.key
```
→ Borra todos los permisos de otros

```
$ chmod a+r README.md
```
→ Todos pueden leer

→ Asignación completa

💡 `chmod -R 755 directorio/` → aplica recursivamente a todo el árbol. Usar con cuidado en /etc o /home.

05 // chown y chgrp — Cambiar propietario

chmod cambia qué se puede hacer. chown cambia quién puede hacerlo.

chown [opciones] nuevo_dueño[:nuevo_grupo] archivo

Solo root puede cambiar el propietario de un archivo.

`$ sudo chown juan archivo.txt`	→ Cambia solo el propietario a juan
`$ sudo chown -R www-data:www-data /var/www`	→ Recursivo: todo el sitio web a www-data
`$ sudo chown juan:developers proyecto/`	→ Cambia dueño Y grupo en un solo paso
`$ sudo chown :docker /var/run/docker.sock`	→ Cambia solo el grupo (dueño sin cambios)

chgrp — Cambiar solo el grupo

```
$ sudo chgrp developers proyecto/

$ sudo chgrp -R www-data /var/www/html
```

Gestión de Usuarios y Grupos

`$ useradd -m -s /bin/bash juan`	Crear usuario con home y shell
`$ passwd juan`	Establecer / cambiar contraseña
`$ usermod -aG sudo juan`	Agregar juan al grupo sudo
`$ usermod -aG docker juan`	Agregar juan al grupo docker
`$ userdel -r juan`	Eliminar usuario y su home

"sudo usermod -aG docker $USER → reiniciá la sesión para que tome efecto."

06 // umask y Permisos Especiales

umask define los permisos por defecto. SUID/SGID/Sticky son las armas secretas del sistema.

umask — Máscara de permisos por defecto

Los permisos finales = Permisos máximos − umask.

Máx. archivos:	666 (rw-rw-rw-)	− 022	= 644 (rw-r--r--)
Máx. directorios:	777 (rwxrwxrwx)	− 022	= 755 (rwxr-xr-x)

```
$ umask              # ver la umask actual

$ umask 027          # más restrictiva (grupo: solo lectura, otros: nada)
```

Permisos Especiales

4 — SUID
s en owner x · ⚠️ Potencial escalación de privilegios

2 — SGID
s en group x · ✅ Útil en entornos colaborativos

1 — Sticky
t en others x · ✅ Esencial en directorios compartidos

Buscar SUID en el sistema (auditoría de seguridad):

```
$ find / -perm -4000 -type f 2>/dev/null   # encuentra todos los SUID del sistema
$ find /srv -perm -2000 -type d 2>/dev/null   # directorios con SGID
$ ls -la /tmp   # verificar sticky bit en /tmp (debería mostrar 't' al final)
```

"find / -perm -4000 es el primer comando que corre un pentester en un sistema Linux."

07 // sudo — Principio de mínimo privilegio

Con gran poder, viene gran responsabilidad. sudo es la forma correcta de escalar privilegios.

¿Por qué NO trabajar como root?

- Un rm -rf mal escrito puede destruir el sistema completo sin advertencia
- Un malware ejecutado como root compromete TODO el sistema de inmediato
- Sin sudo, no hay registro de quién ejecutó qué comandos privilegiados
- Principio de mínimo privilegio: cada proceso solo accede a lo que necesita

```
# /etc/sudoers  (editar SIEMPRE con: visudo)

juan     ALL=(ALL:ALL) ALL           # acceso root total con contraseña
%developers ALL=(ALL) NOPASSWD:/usr/bin/systemctl  # grupo puede usar
systemctl sin pass
deploy   ALL=(ALL) NOPASSWD:/usr/bin/docker        # cuenta deploy controla
docker
```

Comandos sudo esenciales

`$ sudo comando`	Ejecutar como root una vez
`$ sudo -i`	Shell interactivo root
`$ sudo -u www-data bash`	Shell como otro usuario
`$ sudo -l`	Ver mis permisos sudo
`$ sudo !!`	Repetir último cmd con sudo

🛡️ Hardening básico

→ Deshabilitar login SSH de root

→ Usar sudo con logs en /var/log/auth.log

→ Nunca compartir la cuenta root entre admins

"$ sudo grep sudo /var/log/auth.log → audita quién usó sudo y cuándo."

08 // Escenarios del mundo real

Estos son los casos que te vas a encontrar en producción. Resolverlos es el oficio.

Servidor web — NGINX/Apache

Problema: El servicio www-data necesita leer los archivos pero no modificarlos. Los devs necesitan escribir.

```
sudo chown -R www-data:developers /var/www/html
sudo chmod -R 2755 /var/www/html     # SGID para heredar grupo
sudo chmod -R g+w /var/www/html      # grupo puede escribir
sudo chmod 640 /var/www/html/config.php  # config: solo dueño
```

Script de deployment

Problema: Un script debe ejecutarse como root para reiniciar servicios, pero el operador no tiene acceso root.

```
sudo chown root:deploy deploy.sh
sudo chmod 4750 deploy.sh            # SUID: corre como root
# Alternativa más segura: sudoers
deploy ALL=(ALL) NOPASSWD:/usr/bin/systemctl restart nginx
```

Directorio compartido de equipo

Problema: Varios usuarios comparten /srv/proyecto. Cada uno puede borrar solo SUS archivos.

```
sudo groupadd equipo
sudo usermod -aG equipo juan alice bob
sudo chown root:equipo /srv/proyecto
sudo chmod 1775 /srv/proyecto        # SGID + Sticky bit
```

Clave SSH privada

Problema: SSH rechaza la clave si los permisos son muy permisivos. Seguridad crítica.

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_rsa              # clave privada: solo yo
chmod 644 ~/.ssh/id_rsa.pub          # pública: legible por todos
chmod 644 ~/.ssh/authorized_keys
```

"Permisos mal configurados son la causa nº1 de brechas en servidores Linux mal administrados."

LAB 02 // Permisos en la práctica

Abran sus terminales. Construimos un entorno multi-usuario desde cero.

1 Explorar el sistema

```
$ id
$ groups
$ ls -la /home
$ cat /etc/passwd | tail -5
```

Conocer tu identidad: UID, GID y grupos actuales.

2 Crear estructura de trabajo

```
$ mkdir -p
~/lab02/{publico,privado,scripts}
$ touch ~/lab02/privado/secreto.txt
$ touch ~/lab02/scripts/deploy.sh
```

Crear directorios con distintos niveles de acceso.

3 Aplicar permisos octal

```
$ chmod 700 ~/lab02/privado
$ chmod 755 ~/lab02/publico
$ chmod 750 ~/lab02/scripts
$ chmod 600
~/lab02/privado/secreto.txt
```

Asignar permisos correctos a cada directorio.

4 Modo simbólico + ejecutable

```
$ echo "#!/bin/bash\necho Hola CTT" >
~/lab02/scripts/deploy.sh
$ chmod u+x ~/lab02/scripts/deploy.sh
$ ~/lab02/scripts/deploy.sh
```

Hacer ejecutable un script y verificar que corre.

5 Verificar y auditar

```
$ ls -laR ~/lab02
$ stat ~/lab02/privado/secreto.txt
$ find ~/lab02 -type f -ls
```

Auditar permisos del árbol completo con ls y stat.

⭐ Desafío CTT

```
$ sudo groupadd ctteam
$ sudo usermod -aG ctteam $USER
$ sudo chown :ctteam ~/lab02/publico
$ sudo chmod 2775 ~/lab02/publico
$ umask 027 && touch
~/lab02/publico/test_umask
```

Crear grupo compartido con SGID y aplicar umask restrictiva.

⏱️ 50 min · Trabajen en pares · Documenten cada ls -la antes y después de chmod · No usar sudo en los primeros 4 pasos

09 // Cheatsheet — Referencia rápida

Tu tarjeta de referencia para el día a día. Guardá esta slide.

Permisos comunes

- 755 rwxr-xr-x → Directorio / ejecutable público
- 644 rw-r--r-- → Archivo de texto / configuración
- 600 rw------- → Claves privadas / archivos sensibles
- 700 rwx------ → Home / scripts privados
- 750 rwxr-x--- → Scripts para el equipo
- 664 rw-rw-r-- → Archivos colaborativos del grupo
- 777 rwxrwxrwx → ⚠️ EVITAR en producción

chmod — referencia

```
chmod 755 archivo        # octal absoluto
chmod u+x script.sh      # simbólico: add
chmod g-w archivo.txt    # simbólico: remove
chmod o= secreto         # simbólico: clear
chmod -R 755 dir/        # recursivo
chmod 4755 bin/prog      # SUID
chmod +t /tmp            # Sticky bit
```

chown / gestión

```
chown usuario archivo
chown usuario:grupo archivo
chown -R www-data /var/www
useradd -m -s /bin/bash user
usermod -aG grupo user
groups user / id user
sudo visudo              # editar sudoers
```

"man chmod · man chown · man sudoers → La documentación está a un comando de distancia."

FIN DE CLASE 02

Permisos y Usuarios

CTT · Técnico en Redes y Software · Prof. Juan Damián Pajares García

-rwxr-xr-x 2 juan juan 4096 Mar 18

Lo que vimos hoy:

- ✓ Modelo multiusuario: root · usuarios · grupos (/etc/passwd /etc/shadow /etc/group)
- ✓ Leer permisos: ls -la → tipo · owner · group · others · rwx
- ✓ chmod octal (755/644/600) y simbólico (u+x, g-w, o=)
- ✓ chown / chgrp · useradd · usermod -aG · userdel
- ✓ umask: permisos por defecto en archivos y directorios
- ✓ SUID · SGID · Sticky bit — permisos especiales del sistema
- ✓ sudo: principio de mínimo privilegio · /etc/sudoers · visudo

📅 Próxima Clase — Semana 3

Bash Scripting

Automatización · Variables · Condicionales
Bucles · Parámetros · Funciones · Cron

```
$ cat deploy.sh | grep -v '#' | bash -x
```

$ shutdown -h now # ¡Hasta la próxima clase! 🐧 — chmod 755 ~/conocimiento/linux